

SmartFlow Traffic Management System

System Report

Course: INT316D – Web Development Project

Submission Date: 12 May 2026

Student Name	Student Number	Role
Matlala L.B.	231515216	Admin / Users Manager
Moukangwe L.	224139632	Sensors Manager
Magongoa L.M.	221478770	Incidents Manager
Lutchman R.	231141979	Signals Manager

1. Introduction

SmartFlow is a real-time traffic management system designed to help traffic authorities monitor incidents, sensors, traffic signals, and system users. The system provides a centralised web interface where different roles (Admin, Officer, Viewer) can view and update traffic data. The project was developed for a municipal traffic control centre in the Witbank (eMalahleni) area as part of the INT316D Web Development Project module.

1.1 Problem Statement

Traffic management in growing urban areas such as Witbank suffers from several key inefficiencies:

- Delayed incident reporting with no centralised visibility
- Lack of real-time sensor data monitoring
- Manual and uncoordinated control of traffic signals
- No centralised user management or role-based access
- Poor emergency response due to fragmented information systems

SmartFlow addresses these challenges by providing an online platform with role-based access, live data, and full CRUD operations on all key traffic entities.

1.2 Objectives

- Build a 3-tier web application (Client – Web Server – Database)
- Implement CRUD (Create, Read, Update, Delete) operations on four core modules
- Use stored procedures for data consistency and integrity
- Provide a consistent, easy-to-use interface accessible from any browser
- Support role-based access control (Admin, Officer, Viewer)
- Generate summary reports for decision-making

2. System Architecture

SmartFlow follows a 3-tier architecture, cleanly separating the presentation, application logic, and data layers. This design improves maintainability, scalability, and security.

Tier	Technology	Location / Detail
Presentation Tier	Web Browser (Chrome, Firefox) + Thymeleaf Templates + Bootstrap 5	User's device
Application Tier	Spring Boot 3.x + Spring MVC + Spring Security	Laptop A – IP 192.168.8.13 (Windows 10)
Data Tier	MySQL 8.x via JPA / Hibernate ORM	FreeSQLDatabase.com (cloud)

All communication between the web server and database uses secure MySQL connections over the internet. Client devices connect to the Spring Boot web server over the local Wi-Fi network.

2.1 Technology Stack

The following technologies were used across the three tiers:

Category	Technologies
Backend	Java 17, Spring Boot 3.x, Spring MVC, Spring Security (Basic Auth)
Frontend	Thymeleaf Templates, HTML5, CSS3, Bootstrap 5, Font Awesome Icons
Database	MySQL 8.x, JPA / Hibernate ORM, SQL DDL Scripts, Relational Schema (3NF)
Tooling	Eclipse IDE, Git & GitHub, Apache Tomcat (Embedded), Maven Build Tool

3. Database Design

The SmartFlow database is implemented in MySQL and normalised to Third Normal Form (3NF). It consists of five interrelated tables with clearly defined primary and foreign keys.

3.1 Entity Relationship Diagram (ERD)

The five tables and their key relationships are summarised below:

Table	Primary Key	Foreign Key(s)
users	user_id	—
traffic_sensors	sensor_id	—
traffic_logs	log_id	sensor_id → traffic_sensors
traffic_incidents	incident_id	reported_by → users
traffic_signals	signal_id	updated_by → users

3.2 Sample Data

Each table was populated with a minimum of four records to demonstrate full functionality. For example, the users table contains five registered system users with different roles (Admin, Officer, Viewer). The traffic_sensors table lists sensors at key Witbank intersections, including cameras, radar units, and inductive loop detectors. Additional test records such as 'Test 101' and 'Demo Incident' were inserted during development to verify input validation and CRUD operations.

3.3 Stored Procedures

Two stored procedures were implemented to enforce data consistency and automate multi-step operations:

Procedure	Description
sp_ResolveIncident	Updates incident status and sets the resolution timestamp.
sp_AddSensorWithInitialLog	Inserts a new sensor record and automatically creates an initial log entry in traffic_logs, affecting two tables atomically.

4. Features & Modules

4.1 Authentication & Role-Based Access

SmartFlow implements a session-based authentication system with role-based access control (RBAC):

- The login page validates user credentials against the users table in the database.
- On successful login, the session stores the user ID, full name, and assigned role.

- Access to each module is controlled by the `canAccessModule()` function, which evaluates the current user's role before rendering navigation links or allowing access to pages.
- Roles include Admin (full access), Officer (incident and sensor management), and Viewer (read-only access).

4.2 Module 1 – Dashboard

The dashboard serves as the central landing page after login. It displays:

- A personalised welcome banner showing the logged-in user's name and role.
- Four live statistic counters: Total Users (5), Active Sensors (4), Active Incidents (3), Operational Signals (2).
- Quick-link cards to each management module, labelled with the responsible team member.

4.3 Module 2 – Incidents Management (Magongoa L.M.)

The Incidents module provides full visibility and resolution control over traffic incidents across the Witbank road network:

- Displays all incidents in a tabular list showing type, severity badge, location, status, reporting officer, and timestamp.
- Incident types include Accident, Roadwork, Congestion, Hazard, and Weather.
- Severity is categorised as Critical, High, Medium, or Low, each rendered with a colour-coded badge.
- Officers can update the status from Active to Resolved or False Alarm using the `sp_ResolveIncident` stored procedure.
- Non-textbox controls include radio buttons for status selection and a date picker for the resolution date.
- Foreign key dropdown displays the reporter's full name rather than the user ID.

4.4 Module 3 – Sensors Management (Moukangwe L.)

The Sensors module manages the city-wide network of traffic detection devices:

- Add new sensors using the `sp_AddSensorWithInitialLog` stored procedure, which simultaneously creates an initial log entry.
- Delete sensors with cascading removal of associated log records.
- Sensor types include Camera, Radar, and Inductive Loop.
- Statuses include Active, Offline, and Maintenance.
- Non-textbox controls include a range slider for sensitivity threshold and a toggle switch for auto-calibration.
- Dropdowns display sensor type and status as human-readable text rather than database codes.

4.5 Module 4 – Signals Management (Lutchman R.)

The Traffic Signals module allows officers to monitor and update signal behaviour at key intersections:

- Lists all traffic signals with intersection name, operating mode, cycle length (seconds), and current status.
- Supported modes: Normal, Flashing, Manual Override, and Emergency Priority.
- Status options: Operational, Maintenance, and Testing.
- Signal settings are updated via an inline modal form with a status update dropdown.
- Non-textbox controls include checkboxes for pedestrian mode and emergency override, and colour pickers for priority colours.
- Five intersections are currently managed in the system.

4.6 Module 5 – Users Management (Matlala L.B.)

The Users module provides the administrator with full control over system accounts:

- Allows insertion of new users with username, password, first name, last name, email, and assigned role.
- Supports role updates and deletion of user accounts (excluding the currently logged-in account to prevent lockout).
- Non-textbox controls include radio buttons for quick role assignment, a date picker for account expiry, and toggle switches for 2FA, email notifications, and weekly reports.
- Five registered system users are currently active: lmatlala (Admin), lmoukangwe (Officer), lmagongoa (Admin), rlutchman (Admin), tborotho (Viewer).

4.7 Menu & Navigation

A consistent navigation header appears on every page. It includes links to: Dashboard, Incidents, Sensors, Signals, and Users. Login and logout actions are accessible from the header on all pages. Navigation links are conditionally rendered based on the authenticated user's role.

5. Testing

5.1 Test Environment

Component	Details
Web Server	Laptop A – Windows 10, Spring Boot embedded Tomcat
Database	Cloud MySQL via FreeSQLDatabase.com
Client Devices	Other team laptops connected to the same Wi-Fi network (server IP: 192.168.8.13)
Browsers Tested	Google Chrome, Firefox, Edge

5.2 Test Cases

Testing was conducted across all four modules. Each module was verified against the following standard test cases:

- Create: inserting a new record with valid data results in successful database insertion and confirmation message.
- Read: all records are retrieved and displayed correctly in the tabular list view.
- Update: modifying a record's fields (e.g., status, role, signal mode) persists correctly in the database.
- Delete: removing a record updates the list immediately and cascades where applicable (e.g., sensor deletion removes associated logs).
- Validation: submitting a form with missing required fields displays appropriate error feedback.
- Role access: attempting to access restricted pages without the correct role redirects the user appropriately.

5.3 Test Data

All tables were populated with a minimum of four records prior to testing. Additional test data was inserted during development (e.g, 'Test 101' , 'Demo Incident') to stress-test input handling. All foreign key references were verified to be valid, and cascading deletes were confirmed to function correctly.

6. Compliance with Assessment Rubric

The SmartFlow system was developed to meet all criteria specified in the INT316D project marking rubric. The table below summarises how each criterion was satisfied:

No.	Criteria	Points	How the Team Met It
1	Presentation	10	✓ Powerpoint demo having slides with full project walkthrough
2	Functionality	10	✓ Full CRUD implemented on all four core modules
3	Use of Java Framework other than JEE	9	✓ Spring Boot 3.x with Spring MVC and Spring Security
4	Application Server other than Glassfish	10	✓ Embedded Apache Tomcat (via Spring Boot)
5	Upload project to GitHub	5	✓ Repository submitted post-presentation
6	Java IDE other than NetBeans	10	✓ Eclipse IDE used throughout development
7	System Report	6	✓ Architecture, ERD, stored procedures, and modules fully documented
8	Reflection Statement (~1000 words each)	10	✓ All four team members submitted individual reflections
TOTAL		70	

7. Challenges & Solutions

Challenge	Solution
Configuring Spring Security to support role-based access without blocking all pages	Implemented a custom <code>canAccessModule()</code> utility method and used Thymeleaf <code>sec:authorize</code> attributes to conditionally render menu items and pages per role.
Connecting to the remote FreeSQLDatabase.com cloud MySQL instance consistently	Set correct JDBC connection string with SSL disabled and verified firewall rules; used <code>application.properties</code> to manage credentials securely.
Implementing stored procedures that affect multiple tables atomically (<code>sp_AddSensorWithInitialLog</code>)	Used MySQL transactions within the stored procedure to ensure both the sensor insert and log creation either succeed together or roll back on error.
Ensuring consistent UI styling and navigation across all modules developed by different team members	Created a shared Thymeleaf layout fragment with the header, navigation bar, and footer; each module page extends this fragment.
Coordinating development tasks and merging code contributions from four developers	Used a shared GitHub repository with separate feature branches per module; merged to main after peer review and functional testing.
Preventing users from accidentally deleting their own account	Added a server-side check in the Users controller that compares the target <code>user_id</code> against the session <code>user_id</code> before processing a delete request.

8. Conclusion

SmartFlow successfully demonstrates a fully functional 3-tier traffic management web application built with modern Java technologies. The system meets all the requirements of the INT316D project rubric, including a well-structured relational database design, comprehensive CRUD operations across four core modules, stored procedures for data integrity, and a range of UI controls beyond simple text boxes.

The project provided the team with practical experience in full-stack Java web development using Spring Boot and Thymeleaf, collaborative version control via GitHub, and the challenges of building a multi-user, role-based web system. Each team member took ownership of a dedicated module, resulting in a cohesive and functional application delivered within the semester.

The system is ready for presentation and is well-positioned for future enhancements, including real-time map integration for sensor and incident geolocation, automated email notifications for critical incidents, WebSocket-based live dashboard updates, and machine learning-driven automated incident detection from sensor data.